SWALLOOP · RESEARCH BRIEF

How good are Grasshopper's loops?

A technical and UX/UI analysis of Anemone and its peers, a SWOT, and recommendations for Swalloop

Prepared 14 October 2026 for MID 3010 "Loophole" (WiSe 26/27) · Evidence: McNeel Forum, Grasshopper3D archive, Food4Rhino, GitHub, course brief · Evidence grades: **A** author or official statement, **B** user report, **C** inference

1. Summary

0 formal, independent benchmarks of Grasshopper loops found	**~10 yrs** Anemone without a release, until a Package Manager build in Mar 2026	**38.2 s vs 45.2 s** FastLoop vs Python at Count = 400, one user, geometry-heavy	**1** native alternative (GH2 looping clusters), still in beta

Bottom line. Anemone won the community because its mental model is right: a visible *Start* and *End*, nesting, a *while* exit, and results that stay live in a definition. Its weaknesses are structural rather than cosmetic. It works *with* the Grasshopper solver, so every iteration is a full solver lap, and everything that also reacts to the solver (Galapagos, Colibri, clusters, data dams) can fall out of step with it. The code is closed and single-maintainer, and the "fast" variant that users rely on for speed was left out of the 2026 Package Manager build.

- **Efficiency is mostly a question of where the loop is evaluated.** Evaluating on the canvas costs a solver lap per iteration, and the one explicit speed lever users describe is not showing intermediate output ("Output after last"). B In geometry-heavy loops the body dominates and Anemone is on par with a Python script. B
- **The failures people report are about semantics, not speed:** partial data reaching downstream tools, loops restarting inside clusters, off-by-one stops, data-tree flattening, runaway loops with a hard-to-find pause.
- **UX is good at the first hour and weak at the tenth.** Two components read clearly, but status, stopping and modes are hidden or undocumented.
- **The window is real but narrowing.** GH2 ships native looping clusters; GH1 plugins do not load in GH2. A GH1 loop plugin therefore has to be excellent at what native loops will not do first: stopping rules, history, convergence and solver cooperation.

2. Scope, method and what I could not verify

- **Source code.** Anemone is distributed compiled. I found no public repository, and the course brief calls it closed-source. A source-level audit was therefore not possible; statements about internals are inferred from the changelog, forum answers and the Grasshopper solver API (grade C). Hoopsnake has a public repo (`yconst/Hoopsnake`, CC BY-SA 3.0) but the page I could open shows only a compiled `.gha` and a one-line README.
- **Reddit** is blocked for my web tool, so no Reddit threads are included. The McNeel Forum and the archived Grasshopper3D forum carry the same discussions and are used instead.
- **Sources are summaries.** Pages were read through a summarising fetch tool, so wording is paraphrased, not copied. A few items (octopusLoop, the Decodes paper, older Rutten replies) reached me only as search snippets; they are marked.
- **Benchmarks are anecdotal.** Hardware and definitions are mostly unstated. Section 5 proposes a protocol that would replace them.

3. Why Grasshopper has no loops, and how plugins get around it

Grasshopper solves a directed acyclic graph: expired components recompute in dependency order, and a wire that would feed an output back into its own inputs is refused. David Rutten said native loops and recursion were "high on the list" but that clusters had to work first B, and that cycle detection was intended to catch bad wiring B. The three solver ideas that the course names (expiry, scheduling, state) are exactly what every loop plugin recombines:

- **Expiry:** marking an object and everything downstream as needing recomputation.
- **Scheduling:** asking for a new solution once the current one has finished; a callback fires just before it starts.
- **State:** data that survives from one solution to the next.

Anemone's classic loop, as users describe it, keeps the state in Loop Start/Loop End and, when a solution completes, triggers another cycle: *"[it] waits until the current solution completed (all component expired are fully re-calculated), then it will trigger another cycle"* B.

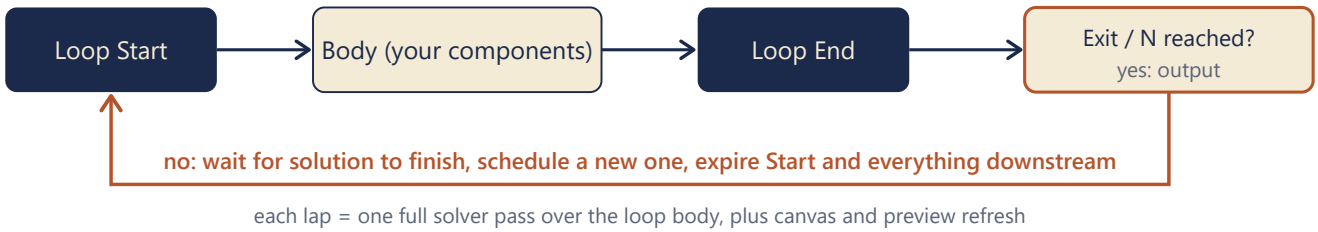


Figure 1. The canvas-evaluated loop model (Anemone classic, Hoopsnake-style). Redrawn from forum descriptions and the solver API notes; grade C.

The alternative is to take the body *off* the canvas and iterate it internally, which is what Anemone's FastLoop appears to do (users report it runs after the loops are done and defaults to "Output after last") and what octopusLoop does explicitly, evaluating the loop in a separate hidden context and claiming up to 1000× speed-ups B (search snippet). The price is solver independence: the loop no longer takes part in the normal expire/solve conversation, which is why FastLoop is the variant that works inside clusters and with Galapagos B, and why it is hard to debug.

4. The landscape

Option	Evaluation model	Strengths (reported)	Limits and status
Anemone classic M. Zwierzycki	Start/End pair; canvas-evaluated, one solver lap per iteration	Intuitive; nested loops; *while* via Exit input; pause; Browse History; expandable data streams; live in a definition B	Closed source; ~10 years without update; tied to solver timing; trees and stop semantics surprise users. In Package Manager since 24 Mar 2026 A
Anemone FastLoop	Iterates internally; "Output after last"	Fastest option in user reports; works in clusters and with Galapagos/Colibri B	Nesting doubted by users; one user measured 5 s for an empty loop (2018); **not in the 2026 package**, reason unanswered A
Hoopsnake yconst	One component copies its input and feeds it back, bypassing the recursion check B	Keeps data order reliably; lets you see each iteration B	Keeps every cycle in memory with no switch to turn that off; reported unsupported; installing it in Rhino 7 drew a 53-reply forum thread B
octopusLoop Vierlinger et al.	Body evaluated in a separate hidden context	Large claimed speed-ups; compatible with Galapagos/Octopus/Goat B	Early stage, nesting listed as a known issue (search snippet; page not opened)
Script components Python / C#	An ordinary `for` loop inside one component	Full control; no solver coupling; "just as easy" for many tasks, per one forum user B	Gives up visual data flow and non-coder access, which is why Anemone exists B
GH2 looping cluster	Cluster routes an output back to an input; *Loop Repeats* pin sets the count	Native; reduces data-tree struggle for many-object cases; recursion-capable per Anemone's author B	GH2 is in the Rhino beta; GH1 plugins do not load; crash reports in the beta thread; release timing speculative B

5. Technical efficiency analysis

5.1 Where the time goes (model)

For a canvas-evaluated loop, total time is approximately `N × (T_solver_lap + T_body + T_refresh)` plus the growth of whatever is recorded. An internally evaluated loop is `N × T_body + T_setup`. This is grade C, but it explains every data point below.

Factor	What the evidence says	Grade
Body cost	Subtracting 400 cones with booleans: Python 45.2 s, FastLoop 38.2 s (July 2025). FastLoop gets relatively faster as Count grows. Hardware and exact file not stated. In this case the geometry, not the loop, is the cost.	B
Fixed per-lap cost	"Fast Loop End" taking over 5 s with no data (May 2018, no cause found). A simple number-series loop at about 1 s per pass (Jan 2019, unresolved). A paper on the Decodes plugin reports ~36 s for 1000 mirror iterations in Anemone against 0.032 s for Decodes (search snippet; paper not opened).	B
Intermediate output	"Output after last" is described by a user as "part of the trick to make it fast".	B
Data growth	Hoopsnake retains every cycle; unmanaged paths that grow by a digit per lap (around 100 digits) slow definitions. Anemone only prepends one integer, and only on recorded loop-end outputs.	B

Referenced geometry	A definition ran slowly with referenced geometry and sped up after re-internalising; a second user reproduced it. Cause not established; likely repeated fetching of Rhino geometry each lap.	B / C
Faster than consumers	A loop produced 2 to 4 times more data than the Kangaroo-based consumer could compute, putting streams out of sync (June 2024).	B

> **Reading.** Loop overhead matters most for cheap bodies and large N (simulation, iteration of arithmetic). When the body is expensive, the plugin barely matters and the architecture question becomes *correctness and cooperation*, not speed. That is where Swalloop can differ most.

5.2 Proposed benchmark protocol

No independent benchmark exists, so one should be created and published with the plugin.

Workload	Body	N	Measured
Trivial	Number update (logistic map, Newton)	100, 1 000, 10 000	Per-lap overhead
Medium	Curve blend / offset, ~50 curves	50, 200, 1 000	Lap cost vs body cost
Heavy	Boolean difference per lap (the 400-cone case)	100, 400	Does loop choice matter?
Coupled	Loop under Galapagos / Colibri, and inside a cluster	fixed	Correctness (data complete?), not time

Compare: Anemone classic, Anemone FastLoop (from the pre-2026 download), Hoopsnake, a Python script, GH2 looping cluster, Swalloop. Record Rhino, CPU, preview on/off, and median of 5 runs.

6. Failure modes users actually hit

#	Symptom	Cause (as understood)	Impact	Evidence
1	Downstream tool captures only the first iteration, or Galapagos "freezes"	Colibri / Galapagos do not wait for the loop; a Data Dam expires nothing, so the loop does not wait for it either. Time Buffer did not help.	High	Colibri threads 2020–2024; data-sync thread Jun 2024
2	Loop restarts or records only once inside a cluster	Clusters solve once per solution, so a canvas loop restarts each time. FastLoop avoids it.	High	Colibri thread; forum reply
3	Output has one extra, invalid value; "No data collected" if the exit is true at the start	The value enters the loop before the exit condition is checked, so the output is the one that failed the test.	Medium	Loop-not-counting thread, Mar 2022
4	Loop stops early or loses branches	A multi-branch tree wired to the single Loop End input is flattened and only the first value kept; invalid self-wiring in "recursion" attempts.	Medium	Forum comments; Recursion thread Jun 2019

5	Loop "never stops" or keeps recording; user cannot find the pause	Exit input is optional, timeout is optional, and pause is a double-click on Loop End that users report not knowing about.	High	Same-data thread Mar 2023; changelog
6	Slow even for trivial bodies	Unexplained per-lap cost; internalisation; component count in the definition.	Medium	Threads May 2018, Jan 2019
7	Nested loops hard to reason about	Pairing and per-level data paths are implicit; users debate whether a list method would be simpler.	Low	Double-loop thread Dec 2025
8	Sharing a definition fails or needs manual unblock	Compiled `.gha` predates the Package Manager; example links were reported broken in 2020.	Medium	"The Anemone problem" Jul 2025; Recursion thread
9	Plugin could break with a Rhino/.NET update	Closed source, one maintainer, "no incentive to update" while it works (his words, Jul 2025).	High	"The Anemone problem"

7. UX / UI analysis

Judged against standard usability heuristics. The two-component layout is a genuine strength; the weaknesses cluster around *status*, *stopping* and *documentation*.

Heuristic	Verdict	Observation
Match to real world	Strong	Start / End reads like a programmer's loop; users call it "more intuitive" than the single-component Hoopsnake and a cleaner layout.
Visibility of status	Weak	The counter is an output, not a state. No built-in "running / converged / stopped by timeout" indicator; an unbounded loop looks the same as a busy one.
User control and freedom	Mixed	Pause, Exit and timeout exist, but pause is a double-click and the timeout is optional, so control is available yet not discoverable.
Consistency	Mixed	Changing Repeats no longer restarts the loop (0.2), which is useful but not obvious; Start and End must have matching data counts.
Error prevention	Mixed	Mismatched Start/End inputs raise an error, which is good. Silent flattening of trees and the off-by-one stop are not caught.
Recognition over recall	Weak	Loop modes (fast, internalized), Time Buffer, Record and Output-after-last are named in the changelog but not explained on the project page.
Debuggability	Mixed	Browse History (0.26) helps; Hoopsnake users valued seeing every iteration. No per-iteration timing or convergence view.
Help and documentation	Weak	Changelog-style documentation, example files hosted off-site, license and source status unstated.
Installation and sharing	Improving	Manual unblock and install for a decade; Package Manager build since March 2026, classic components only.

8. SWOT

Subject: the existing Grasshopper 1 loop ecosystem, with Anemone as the reference implementation.

Strengths

- Right mental model: visible Start/End, nesting, *while* via Exit
- Live inside a definition; re-runs when upstream changes
- Pause, timeout, history browsing present
- Competitive with Python on geometry-heavy bodies (38.2 s vs 45.2 s at N = 400, one user)
- Large, helpful community and a decade of examples
- Reported as dependable: "has never failed me"

Weaknesses

- Closed source, single maintainer, ~10 years without release
- Canvas evaluation couples the loop to solver timing
- Fixed per-lap cost; unexplained slowdowns
- Surprising semantics: off-by-one stop, tree flattening, recording paths
- Hidden controls (pause) and under-documented modes
- FastLoop not in the 2026 package
- Manual install friction for most of its life

Opportunities

- An open, tested, documented GH1 loop plugin: the gap the course brief itself names
- First-class convergence, history and status that none of the options offer
- Explicit solver cooperation (a "done" signal) for Galapagos and Colibri workflows
- One semantic with two execution modes (inspect vs run)
- Package Manager, example files and a published benchmark as differentiators
- GH1 users stay on GH1 for production while GH2 is in beta

Threats

- GH2 native looping clusters may absorb the common use cases
- GH1 plugins do not load in GH2: the ecosystem is splitting
- Rhino or .NET updates can break an unmaintained compiled plugin
- Python/C# loops are "just as easy" for many users, and AI assistants write them on demand
- Fast, internal evaluators (octopusLoop) set a high speed expectation
- Reputation: a new loop plugin inherits users' fear of hangs and lost data

9. Recommendations for Swalloop

Ordered by how much they change the architecture, so the first group should be decided *before* the team's shared repository is scaffolded. Items marked ✓ are already in the prototype (`swalloop_erst`) and should be carried over as requirements, not re-invented.

Decide now (architecture)

#	Recommendation	Why, and what it fixes	Failure modes
1	**One loop semantic, two execution modes.** *Inspect*: canvas-visible, per-iteration. *Run*: internal, output-after-last. A test asserts identical results.	Gives Anemone's debuggability and FastLoop's speed without two plugins and without users guessing which to trust.	5, 6

2	**Define stop semantics once, in writing.** ✓ Condition evaluated at the start of each pass on the current state; ✓ tolerance in the data type's own units; return a *status* (converged, max reached, condition, cancelled).	Removes the off-by-one family and lets downstream logic react to *why* a loop ended.	3, 5
3	**Solver cooperation contract.** A "done" output that is false until the loop finishes, correct expiry so that Galapagos / Colibri / data dams wait, and a documented pattern for clusters. Test with Galapagos in CI.	The most-reported real-world breakage; nobody currently solves it deliberately.	1, 2
4	**Host-independent core, thin GH1 adapter.** ✓ The engine already knows nothing about towers; extend that to "knows nothing about Grasshopper".	Makes a GH2 port and unit tests possible, and answers the maintainability threat.	9

Build into the component (UX)

#	Recommendation	Why, and what it fixes	Failure modes
5	**Safe by default.** A finite default Max iter, a Stop button on the component face (not a hidden double-click), and an optional wall-clock timeout.	An unbounded loop must never be one slider move away.	5
6	**Visible state.** Face shows lap counter, change per lap, and status badge (running, converged, capped). Show "Loop · number" type as the prototype does ✓.	Replaces guessing with reading.	5, 6
7	**Explicit data-tree contract.** No silent flattening; paths preserved; branch index added only when Record is on, and documented. Typed wires ✓ that turn red when types stop fitting.	Targets the "lost branches" and growing-path complaints.	4
8	**History as a typed output.** ✓ `{ frames, count, max }` plus per-iteration timing, and a convergence curve in the inspector ✓.	Hoopsnake users valued iteration visibility; make it default, not an add-on.	6
9	**Errors that say what to do.** "Loop End has 3 data inputs, Loop Start has 2. Add one input to Start." Warn when a tree is wired into a single-value input.	Turns silent failures into guidance.	3, 4

10	**Nested pairing you can see.** ✓ Nearest unclaimed Start upstream; additionally highlight the paired Start when End is selected.	Reduces the implicit-pairing confusion.	7

Release engineering (the course's fifth subject)

- **Ship through the Package Manager from the first release**, with semantic versioning, an open licence stated in the README and manifest, and example files *inside* the package (no off-site links).
- **Publish the benchmark** from Section 5.2 with the release, including cases where Swalloop is not the fastest. Credibility is the main asset a loop plugin lacks.
- **Document every mode and option on the page where users read first**, not only in a changelog; the prototype's README table of loop kinds is a good template.
- **Name and brand with intent.** The swallow metaphor (take-off, return, migration, nest) can carry component names, tooltips and icons consistently, which also helps recognition over recall.

10. Open questions to validate

1. How does FastLoop actually iterate (own sub-document, or re-entrant solve)? Decompiling is a licence question; asking the author or measuring behaviour is safer.
2. What is Anemone's real per-lap cost on Rhino 8 with the 2026 build? A benchmark run (Section 5.2) answers this for the classic components.
3. Does a GH1 plugin gain anything by also exposing its loop as a cluster-compatible component for GH2 later?
4. Which of failure modes 1 and 2 can be fixed in GH1 without private APIs?

Sources

Grades: A author or official, B user report, C inference. Items marked † were read only as search snippets or could not be opened.

1. Anemone group page and changelog, Grasshopper3D. grasshopper3d.com/group/anemone
2. "The Anemone problem", McNeel Forum, Jul 2025 to Mar 2026 (maintainer statements, benchmark, Package Manager release). discourse.mcneel.com/t/the-anemone-problem/206615
3. "Anemone very slow, when data is not internalized", Jan 2019. discourse.mcneel.com/t/.../77008
4. "Anemone's 'Fast loop end' is... slow", May 2018. discourse.mcneel.com/t/.../63573
5. "Combining Anemone and Colibri", 2020 to 2022. discourse.mcneel.com/t/.../99181
6. "Data sync issue in Anemone", Jun 2024. discourse.mcneel.com/t/.../184544
7. "Anemone Loop not counting right?", Mar 2022. discourse.mcneel.com/t/.../139384
8. "Anemone loop same data", Mar 2023. discourse.mcneel.com/t/.../156645
9. "Recursion in Anemone", Jun 2019. discourse.mcneel.com/t/.../84571
10. "Double loop with anemone", Dec 2025. discourse.mcneel.com/t/.../213841
11. "Anemone or other plugin to do grasshopper procedural loops in Rhino 8?", Jan 2025. discourse.mcneel.com/t/.../198464
12. "Rhino Beta Feature: Grasshopper 2 (Beta)", Jul 2026. discourse.mcneel.com/t/.../221336
13. "Looping Cluster Example for Grasshopper2", Nov 2025. discourse.mcneel.com/t/.../212536
14. Hoopsnake repository, `yconst/Hoopsnake`. github.com/yconst/Hoopsnake
15. † Hoopsnake vs Anemone comparison comments, Grasshopper3D archive (search snippets). ...Comment:989099, ...Comment:1429486
16. † octopusLoop, Food4Rhino (page returned 403; description via search results). food4rhino.com/en/app/octopusloop
17. † "Iterative Pattern Design via Decodes Python Scripts in Grasshopper" (benchmark via search snippet). academia.edu/43110913
18. † D. Rutten replies on solver scheduling and loop plans, Grasshopper3D archive (search snippets). ...Comment:918643

19. MID 3010 course brief "Loophole", WiSe 26/27 (project_definition.pdf, updated 13 Oct 2026).